

CommerceOps AI (RTOShield) – Product Vision & Architecture

Executive Summary: CommerceOps AI’s RTOShield is an operations control-room for Indian D2C brands, aimed at recovering profit leaks from Cash-on-Delivery (COD) returns and failed deliveries (RTO/NDR). It shifts RTO from a post-facto metric to a leading KPI 【12†L45-L54】. By orchestrating data-driven workflows – order risk scoring, address validation, proactive notifications, and exceptions handling – the platform saves tens of thousands of rupees per month for a 2,000-orders/mth seller. For example, reducing RTO from 28% to 22% on 4,000 COD orders saved ~₹57,600–89,760 【14†L236-L244】; proportionally, a 2,000-order seller can recover ~₹30k–45k monthly. Key features (Action Queue, Risk Scoring, NDR Rescue, Prepaid Offers, Pincode/Courier analytics, etc.) are prioritized by ROI. The architecture is fully modular (microservices, event-driven) with pluggable connectors (WhatsApp API, courier APIs, Shopify, payments). A central feature-registry enables toggling modules by plan/role (using a disciplined feature-flag pattern 【24†L77-L82】). The UX provides a “Profit Cockpit” dashboard, task queues, rescue screens and reports, all configurable per workspace mode. Privacy complies with India’s DPDP rules (e.g. 3-year retention for user data 【27†L533-L541】). Comprehensive testing (unit, contract/integration, and targeted E2E) and CI checks ensure quality 【29†L121-L129】. In summary, RTOShield is a data-driven, API-first SaaS that converts operational action into profit, with clear implementation guidelines ahead.

Product Vision & Positioning

CommerceOps AI positions itself as an **AI-driven e-commerce control room** for profit optimization. The RTOShield module tackles India’s core pain: COD-heavy RTO losses. In India, 15–20% of orders face delivery exceptions and ~30–40% of those become RTOs 【22†L118-L121】. Each RTO order implies a “triple-layered” loss – lost sale revenue plus forward and return shipping costs 【22†L176-L184】. Traditional dashboards hide this hidden cost; RTOShield makes it visible and actionable. Its vision is to flip RTO from a “logistics afterthought” into a strategic KPI 【12†L63-L71】. We integrate across the stack (orders, messaging, courier, payments) to prevent waste at *every* step: verifying risky orders (before dispatch), fixing addresses, converting COD to prepaid, automating reattempts, and analyzing structural leaks (pincodes, products, couriers). In essence, RTOShield is a command-center for D2C ops: it frees brands to focus on growth by automating loss-prevention, turning each saved delivery into pure margin.

Key positioning points:

- **Profit-first, not volume-first:** highlights how RTO drains profit 【12†L45-L54】.
- **Action-driven automation:** drives workflows (calls, messages, holds) to *prevent* RTO in real time.
- **Modular & flexible:** features are plug-ins enabled per plan/role with feature toggles 【24†L77-L82】.
- **Data & AI powered:** uses analytics (rule-based now, ML later) to predict and score risk and opportunities.
- **Indian-focused:** optimized for India’s COD/RTO realities (e.g. local courier patterns, WhatsApp usage).

Prioritized Feature Map (Must/Should/Optional)

Each feature is prioritized by impact (money saved) and customer value. **Must-haves** deliver direct ROI; **Should-haves** enable scale or improve efficiency; **Nice-to-haves** support sales/UX. Savings estimates assume 2,000 orders/month with ~70% COD and a 24% RTO baseline (≈480 RTO orders). We assume ~₹150–200 loss per RTO order (shipping + lost margin) 【14†L146-L154】. Savings are approximate ranges (₹/mo) for that profile.

Feature	Category	Justification & Mechanism	Est. Savings (₹/mo)
Order Import & Data Quality	Must	Clean CSV ingestion (mapping, validation). Ensures up-to-date data for all ops; bases analytics on accurate orders.	₹0 (foundation)
Rule-Based Risk Scoring	Must	Flags high-risk COD orders (bad address, new customer, remote pincode). Prevents shipments likely to RTO.	₹10k–25k
COD Prepaid Incentive Offers	Must	Identifies high-risk COD orders to convert to prepaid with safe discount. Reduces COD RTO on those orders.	₹5k–15k
Address Validation & Correction	Must	Detects weak/incomplete addresses (e.g. missing landmark) pre-shipment and prompts fixes via customer. Reduces failed deliveries.	₹5k–15k
Daily Action Queue	Must	Aggregates all pending tasks (confirm COD, call customer, resend NDR, follow-ups) in one list. Ensures timely handling of RTO causes.	₹10k–25k
NDR Rescue Workflow	Must	Tracks failed delivery attempts (NDRs) and automates re-attempt strategies (calls/SMS/WhatsApp). Recovers sales before RTO.	₹5k–15k
NDR Playbooks	Should	Provides scripted actions per NDR reason (Unavailable, Refused, WrongAddress, etc.), optimizing recovery strategy.	₹3k–10k
Pincode Intelligence	Must	Analyzes RTO/COD rates by pincode, recommending policy (hold/block COD, switch courier). Prevents shipments to persistently bad areas.	₹10k–30k
Courier Intelligence	Must	Identifies high-failure courier/pincode lanes, recommends alternate couriers. Cuts courier-caused RTO.	₹5k–15k
Policy Simulator	Should	Models changes (e.g. hold X COD, disable Y pincodes) to estimate impact. Helps founders choose optimal policies.	₹2k–8k
Custom Rules Engine	Should	Enables business-defined rules (hold threshold, dynamic incentives). Allows fine-grained control.	₹2k–8k
Prepaid			

Opportunities Report	Should Category	Justification & Mechanism	₹3k–10k Est. Savings (₹/mo)
SKU/Campaign Leakage Report	Should	Highlights products or ad campaigns with unusually high RTO. Enables marketing & catalog fixes.	₹2k–10k
Savings Ledger & ROI Dashboard	Must	Logs all “avoided loss” events (cancellations, rescues) and shows net benefit. Proves value (not direct savings but crucial for adoption).	₹0 (proof of ROI)
Profit Cockpit Dashboard	Must	Executive view of top KPIs (loss avoided, urgent actions, worst pincodes/couriers). Focuses team on highest-leverage items.	₹0 (facilitator)
Weekly/Monthly Reports	Should	Automated executive reports (trend summaries, top actions, strategic insights). Aids retention and strategy.	₹0 (process support)
14-Day Pilot Workflow	Optional	Guided trial setup & review for new clients. Drives initial data import and quick wins.	₹0 (sales tool)
Audit Flow (RTO Calculator)	Optional	Pre-sales analysis: estimated RTO losses and savings. Builds trust.	₹0 (lead gen)
Brand/Plan Settings	Must	Configuration of margins, shipping costs, COD fees, plan entitlements. Needed for calculations.	₹0 (enabler)
Stores & Integrations Setup	Should	Manages multi-store mapping and connector config (Shopify, Woo, APIs). Ensures data pipelines.	₹0 (enabler)
Privacy & Audit Logs	Must	Masks personal data, logs user actions, supports GDPR/DPDP compliance. Builds trust.	₹0 (compliance)

These estimates assume conservative mid-tier impact. For instance, [14] shows an 8pp RTO reduction on 4,000 COD saves ~₹57k–90k 【14†L236-L244】. Pro-rating to 2,000 orders suggests ~₹30k–45k from a similar RTO improvement. Each feature above targets a slice of that: e.g. Risk Scoring + Action Queue might collectively drop RTO by 5–8pp (≈₹15k–25k saved), while Pincode/Courier analytics address systemic issues (another ₹10k–30k). The Savings Ledger/ROI tools don’t directly save money but are critical to monetize the solution by proving savings to the client.

Modular System Architecture

We recommend an **event-driven microservices architecture** (EDMA) for flexibility and scalability 【18†L310-L319】. Each feature group runs in a loosely-coupled service (module) behind the UI. Services communicate via a central event bus (e.g. Kafka, AWS SNS/SQS) so that one action (e.g. *Order Imported*) can trigger downstream processing (risk scoring, stats updates, action creation) asynchronously. This pattern ensures resilience and easier maintenance: “applications built as a collection of loosely coupled services... more distributed, faster to build, easier to scale, and more reliable” 【18†L310-L319】.

Key components (and example files/names):

Service/Module	Responsibility	Key Files/Interfaces
Core Service	User auth/roles, brand/store config, feature flag registry	core/ , e.g. FeatureRegistry.ts , UserStore.ts
Order Service	Ingest & store orders, compute risk scores, address check	orders/ - OrderImporter.ts , Order.ts , RiskScorer.ts
NDR Service	Track NDR events, playbooks, retry logic	ndr/ - NdrCase.ts , PlaybookEngine.ts
Action-Queue Service	Manage tasks (confirm COD, call, etc.), queue and completion	actions/ - ActionQueue.ts , ActionItem.ts
Messaging Service	Send/receive notifications (WhatsApp/SMS), template management	messaging/ - WhatsappConnector.ts , TemplateStore.ts
Savings Ledger Service	Record savings events (cancellations, rescued NDR, etc.)	savings/ - SavingsEvent.ts , Ledger.ts
Pincode & Courier Stats Service	Aggregate stats (RTO/COD rates by pincode/courier)	stats/ - PincodeStats.ts , CourierStats.ts
Report Service	Generate dashboards, weekly/monthly PDF or email reports	reports/ - DashboardGenerator.ts , ReportScheduler.ts
Integration Connectors	External APIs: WhatsApp BSP, Courier APIs, Shopify, Payment	connectors/ - ShopifyApi.ts , CourierApi.ts , PaymentLink.ts

Event Bus (Kafka/SNS) Service/Module	Pub/Sub backbone for inter-service events (OrderCreated, ActionCompleted, etc.)	p/a (infrastructure) Key Files/Interfaces
---	---	--

Each service exposes REST/GraphQL APIs (e.g. `GET /api/orders` , `POST /api/actions/{id}/complete`) and subscribes/publishes domain events. For example, `OrderImported` → triggers risk scoring and stats update; `ActionTriggered` → may send a Whatsapp message; `NdrResolved` → generates a `SavingsEvent` . This decoupling aligns with best practices for complex domains **【18†L310-L319】** .

Canonical Data Model

Below is a simplified schema of the core entities (key fields and types). Fields like `userId` , `brandId` , etc. are UUIDs. This model is an example – actual schemas may include timestamps, indices, etc.

Entity	Key Fields (Type)	Description
Order	<code>id: UUID</code> , <code>storeId: UUID</code> , <code>createdAt: datetime</code> , <code>value: number</code> , <code>codAmount: number</code> , <code>status: enum</code> (New/Packed/Shipped/Delivered/RTO) , <code>rto: bool</code> , <code>address: string</code> , <code>pincode: string</code> , <code>courier: string</code>	Customer order record (after import). Tracks COD, status, RTO.
ImportJob	<code>id: UUID</code> , <code>filename: string</code> , <code>importedAt: datetime</code> , <code>status: enum</code> (Pending/Success/Failed) , <code>rowCount: int</code>	Records a CSV import batch.
NdrCase	<code>id: UUID</code> , <code>orderId: UUID</code> , <code>attempts: int</code> , <code>reasonCode: string</code> , <code>status: enum</code> (Open/Resolved/RTOed) , <code>createdAt: datetime</code>	A failed-delivery event for an order.
ActionItem	<code>id: UUID</code> , <code>orderId: UUID</code> , <code>type: enum</code> (Call, WhatsApp, Hold, Cancel) , <code>status: enum</code> (Open/Done/Skipped) , <code>assignedTo: UUID</code> (user) , <code>dueAt: datetime</code>	A task in the daily queue (e.g. call customer).
SavingsEvent	<code>id: UUID</code> , <code>orderId: UUID</code> , <code>type: enum</code> (Cancelled, NDRRescue, PrepaidConverted) , <code>amountSaved: number</code> , <code>timestamp: datetime</code>	Logged “avoided loss” events (used in Savings Ledger).
PincodeStats	<code>pincode: string</code> , <code>month: string</code> , <code>totalOrders: int</code> , <code>codOrders: int</code> , <code>rtoOrders: int</code>	Aggregated metrics by pincode & time (for reporting).
CourierStats	<code>courier: string</code> , <code>month: string</code> , <code>totalOrders: int</code> , <code>rtoOrders: int</code>	Aggregated metrics by courier & time.
PrepaidOpportunity	<code>id: UUID</code> , <code>orderId: UUID</code> , <code>recommendedIncentive: number</code> , <code>probSuccess: number</code>	Suggests converting a COD order to prepaid.
User	<code>id: UUID</code> , <code>brandId: UUID</code> , <code>email: string</code> , <code>name: string</code> , <code>role: enum</code> (Admin/Founder/Ops/Analyst/Viewer) , <code>passwordHash: string</code>	Platform user account.
Brand	<code>id: UUID</code> , <code>name: string</code> , <code>settings: JSON</code>	D2C brand profile (shipping costs, COD fees, etc.).
Store	<code>id: UUID</code> , <code>brandId: UUID</code> , <code>platform: string</code> , <code>apiCredentials: JSON</code>	Linked e-commerce store (Shopify, WooCommerce, etc.).

This canonical model ensures that orders, NDR cases, actions and analytics are consistently recorded. For example, the *RTO rate* formula is simply `(rtoOrders / totalShipped)×100` **【12†L75-L79】** . Calculated fields (riskScore, incentives) can be derived at runtime or stored as needed.

Key Algorithms & Heuristics

All core logic begins as deterministic rules/metrics; ML placeholders can be added later. Examples:

- Risk Scoring Rule:** For each COD order, compute a risk score by summing rules (e.g. pincode known RTO hotspot, no landmark, first-time customer, high COD value, etc.). E.g.: `if (addressWeak) score+=2; if (orderValue>1000) score+=1; if (firstOrder) score+=1` . Above threshold, create a *Hold* action or flag for review.
- RTO Loss Calculation:** `estimatedRtoLoss = orderCount * (forwardShipping + reverseShipping)` **【14†L146-L154】** . With average ~₹65–85 forward and ₹55–75 return, typical loss is ₹120–160 per RTO order **【14†L146-L154】** . The system uses brand shipping settings or courier rates to compute each order’s shipping.
- COD Prepaid Opportunity Scoring:** For each COD order, calculate `netSavings = (CODFee - prepaidDiscount)` . Offer an incentive (e.g. ₹X wallet cashback) if `netSavings > 0` . Adjust for product margin: ensure `(orderValue - incentive) * marginPct > (orderValue * marginPct - costOfRTO)` . In short, only incentivize when it increases net profit.
- NDR-to-RTO Risk:** If an NDR case is *unresolved* after 1st attempt and the order value (or COD amt) is high, escalate urgency or auto-book reattempts. Use time thresholds (e.g. if >24h since NDR, send escalation message).
- Pilot Outcome Scoring:** To measure pilot success, compare `baselineRtoRate` vs `currentRtoRate` and compute improved contribution. (e.g. an 8-point drop can yield ~28% profit bump **【14†L259-L264】**).

We store placeholder hooks for future ML models: e.g. “RiskScoreModel.predict(order)” or “RTOProbModel.predict(order)”. For now, rule-based approaches ensure determinism and easy explainability.

UX / Design System

The UI is a single-page app (likely React/TypeScript) with a clean, modular layout. Primary navigation (sidebar) is dynamically built from the **Feature Registry** and enabled features (per plan/role). Key UI sections:

- **Profit Cockpit (Home)**: Hero metrics (total orders, COD rate, RTO rate, estimated leakage, net savings). Visual KPIs (cards, graphs) highlight urgency. Use color-coded badges (red for critical RTO, green for savings) and sorted priority tasks. For example, show: “*Estimated Preventable Loss: ₹X*”, “*Top Actions: Y calls, Z messages*”.
- **Daily Action Queue**: List of pending actions (filters: overdue/critical first). Each ActionItem component shows order summary (customer, pin, COD amt), recommended actions (☑ confirm COD, ☑ call, ☑ WhatsApp, etc.), and a priority badge. Actions are grouped by type.
- **NDR Rescue**: Table of active NDR cases. Columns: OrderID, last attempt date, NDR reason, attempts, promised delivery, actions (Call, Reschedule, Cancel). SLA timers (red highlight if > threshold). Ability to log results.
- **Prepaid Opportunities**: List of COD orders flagged as high-value that can be offered incentives. Shows order details, recommended incentive, potential savings. Actions: “Send Prepaid Offer” or “Skip”.
- **Reports & Intelligence**: Interactive dashboards and charts. Pincode heatmaps (RTO%), courier comparison tables, SKU leakage charts, and campaign ROI if UTM data present. Export or PDF-printable for weekly/monthly strategy reports.
- **Feature Registry & Toggles** (Admin settings): A settings page listing all features (with descriptions, plan requirements, toggle switch). Features are grouped (Core, Analytics, Workflows, Reports, Integrations, etc.). Toggling a feature on/off immediately shows/hides corresponding menu items and UI modules (per the toggling pattern `【24tL77-L82】`).
- **Workspace Modes**: Onboarding or a switcher allows choosing “Founder Mode, Ops Mode, Analyst Mode, Pilot Demo Mode”. Each mode presets a dashboard layout (e.g. Founder sees high-level Cockpit + weekly report; Ops sees Action Queue + NDR list). This is a UI-layer personalization on top of feature toggles.
- **Dashboard Widgets**: The homepage dashboard can be customized (drag or toggle widgets): e.g. toggle COD exposure chart, Courier Stats, Action Preview, etc. Widget components (MetricCard, ChartPanel, PriorityList) accept props like `title`, `dataSource`, `priority`, `size`.

All UI components should be designed for clarity and responsiveness (e.g. use standard design system). Empty states and tooltips explain metrics (e.g. “RTO Rate = returned orders / shipped `【12tL75-L79】`”). Critical actions (e.g. disabling a core feature) prompt confirmation. The onboarding flow guides new users: select workspace mode, connect store, upload initial CSV or demo data, and automatically enable recommended feature bundle.

Integration & Connectors

RTOShield defines clear **connector interfaces** to external systems. At launch, we use **mock providers** (simulated SMS/WhatsApp, fake courier APIs) for development, with stubs implementing the same interfaces. Key integrations:

- **WhatsApp Business Platform**: We design a `WhatsappService` interface with methods like `sendTemplateMessage(phone, templateName, variables)`. Use a mock in dev; in prod integrate with an official BSP (Meta API). (WhatsApp charges ₹0.1150 per transaction/utility message in India `【10tL230-L237】` – we’ll classify our messages as “utility” when triggered by user actions to minimize cost.) Free service messages inside 24h windows are leveraged.
- **SMS/Email Gateway**: Similar interface for SMS and email, for redundancy or non-WhatsApp users.
- **Courier APIs**: `CourierConnector` interface to subscribe to NDR webhooks (courier notifications), to send “re-attempt” or “reschedule” requests if supported, and to fetch tracking. Each courier (Delhivery, XpressBees, etc.) adapter implements standard calls.
- **E-commerce Platforms**: `StoreConnector` for Shopify/WooCommerce – to periodically fetch new orders (or webhooks) and to update order status (e.g. mark “Rescheduled”).
- **Payment Links**: Interface to payment providers (e.g. Razorpay, Paytm) to generate a payment link for COD-to-prepaid conversions.
- **Event Bus**: Internally, events like `OrderCreated`, `NdrCreated`, `ActionCompleted`, `SavingsRecorded` are published. These topics (Kafka topics or SNS topics) decouple services and allow new features (or ML models) to subscribe later without changing existing flows.

Each connector is mocked/tested in staging. Rate limiting and retries are built into adapters. For example, WhatsApp adapter will handle template approvals and retry failures; courier adapters must handle partial failures (logging them as NDRs if needed).

Privacy, Data Retention & Security

Compliance and security are integral. Personal data (customer names, phones, addresses) is stored securely: encrypted at rest and in transit (e.g. AES-256, TLS). Access to PII is limited by role (Ops vs Analyst vs Viewer). All access/actions are logged (audit trail per user).

We follow India’s DPDP Act 2023 rules `【27tL533-L541】`: e-commerce platforms must delete personal data after certain retention. Draft rules indicate a *3-year* retention for large sites `【27tL533-L541】`, so we design for configurable retention policies (e.g. auto-deletion of imported data older than X years). We explicitly collect only needed consent. Data sharing (e.g. with couriers or WhatsApp) abides by opt-in and encryption.

Security measures:

- OAuth2/JWT for user auth, with granular RBAC (Admin/Founder/Ops/Analyst/Viewer).
- Strong input validation to avoid injection.
- Regular vulnerability scanning and secrets management.
- For production, host on a secured cloud (with firewalls, VPC) and use managed DBs with backups. Privacy note: Phone numbers should be masked in the UI (e.g. show only last 4 digits) unless Ops explicitly unlocks for calling. Records auto-delete/archival after retention period.

Testing & CI Strategy

A thorough testing pyramid is enforced.

- **Unit Tests**: Each service/module has extensive unit tests for business logic (risk scoring, action creation, formula calculations). This covers ~70% of code.
- **Integration Tests**: Services are tested together with in-memory or test databases. Key flows (e.g. importing orders triggers actions) are tested end-to-end via contract tests.
- **End-to-End (E2E) Tests**: Critical user journeys (upload CSV → view dashboard, mark action complete; NDR triggers generate action) run in a staging environment. We keep these focused to avoid flakiness – for example, one E2E test to ensure “creating an NDR leads to a WhatsApp send and a Task in queue”. Well-designed E2E suites are a “safety net” that catch multi-service issues `【29tL121-L129】`.
- **CI/CD**: Every commit/PR runs linting, unit tests, and integration tests. Nightly builds deploy to a staging environment where E2E scripts run (CI agent). Code coverage and dependency checks are automated.
- **Acceptance Criteria**: For each feature, we define test cases. Example: toggling off “NDR Rescue” must remove NDR menu and prevent any NDR actions. Converting an order to prepaid should mark the order accordingly and create a ledger entry. We use Gherkin or similar for user stories where helpful.

Key quote: Even in microservices, E2E tests ensure all parts “work together as a cohesive system” and catch cross-service regressions 【29†L121-L129】. We apply this principle to the most important flows (orders through to savings recording). For UI, we use component-level tests (e.g. verify the dashboard shows correct badges given mocked data).

Deployment & Acceptance Criteria

We package each service as a Docker container. Environments:

- **Local/Dev:** Docker Compose with service stubs, or a lightweight Kubernetes (e.g. Kind). Developers can spin up all modules, inject demo data, and interact via the UI or API.
- **Staging:** Full k8s cluster (AWS EKS/GKE) mirroring prod config (minus real credentials). Run CI pipelines, smoke tests, and on-demand user testing.
- **Production:** Kubernetes with scaling. Secrets in vault. We use blue-green or canary deploys for safe releases.

Deployment pipeline:

1. Code merged → Build artifacts.
2. Unit/integration tests run.
3. Deploy to QA/staging → run E2E tests.
4. Manual QA sign-off (with sample data on demo account).
5. Promote to prod.

Acceptance Criteria (example):

- *Core functionality:* After importing 1,000 mock orders (via CSV or Shopify), the dashboard must display correct COD/RTO rates, actions queue populated, and reports accessible.
- *Modularity:* Switching Plan from Starter to Growth toggles on advanced features (policy simulator, playbooks) and UI updates immediately.
- *Performance:* The system handles 2,000 orders and 500 actions without slowdown.
- *Security:* No plaintext PII in logs; unauthorized API calls are rejected.
- *Privacy:* User data older than X years is either purged or anonymized according to settings.

Any change that breaks a core workflow (e.g. an order not triggering risk scoring) fails CI. We consider these as “Definition of Done” for features.

Tables:

Table 1. Prioritized Features & Savings (for 2k orders/mth)

Feature	Category	Est. Savings (₹/mo)	Rationale
Risk Scoring & High-Risk Hold	Must	₹10,000–25,000	Avoids shipping orders likely to return.
Prepaid Conversion Offers	Must	₹5,000–15,000	Converts high-risk COD to prepaid, saving on RTO.
Address Quality Check & Fix	Must	₹5,000–15,000	Prevents failed deliveries from bad addresses.
Daily Action Queue	Must	₹10,000–25,000	Ensures timely RTO/NDR resolution and follow-ups.
NDR Rescue Workflow	Must	₹5,000–15,000	Automates outreach on failed deliveries.
Pincode Risk Analysis	Should	₹10,000–30,000	Blocks or re-routes COD in chronic RTO zones.
Courier Performance Insights	Should	₹5,000–15,000	Switches away from underperforming lanes.
Custom Rules Engine	Should	₹2,000–8,000	Fine-tunes policies (e.g. hold all COD >X).
Policy Simulator	Should	₹2,000–8,000	Quantifies impact of new COD/RTO policies.
SKU/Campaign Leakage Reports	Optional	₹2,000–10,000	Identifies product or marketing issues causing RTO.
Weekly/Monthly Strategy Reports	Optional	₹0 (support)	Provides executive insights (retention tool).
Demo & Onboarding Bundle	Optional	₹0 (sales/UX)	Drives adoption via guided experience.

Table 2. Core Services/Modules & Files

Service / Module	Responsibility	Key Files/Endpoints
Core/Auth Service	User auth, RBAC, feature-flag registry	AuthController , FeatureRegistry.ts
Order Service	Ingest/orders DB, risk scoring, address check	OrderImportHandler , RiskScorer.ts
NDR Service	NDR case tracking and resolution logic	NdrTracker.ts , PlaybookEngine.ts

Action Queue Service / Module	Task list management (calls, holds, etc.) Responsibility	ActionController, ActionQueue.ts Key Files/Endpoints
Messaging Service	WhatsApp/SMS integration, template management	WhatsappConnector.ts, SmsSender.ts
Savings/Ledger Service	Records and reports savings events	SavingsEvent.ts, Ledger.ts
Analytics Service	Aggregates stats (pincode/courier/sku metrics)	PincodeStats.ts, CourierStats.ts
Report Service	Dashboards, PDF/Excel export	ReportGenerator.ts, Dashboard.ts
Connectors	External API adapters (Shopify, courier, etc.)	ShopifyApi.ts, CourierApi.ts
Event Bus	Pub/Sub backbone (Kafka topics, etc.)	N/A (infra)

Table 3. Data Model Schema (Entities & Fields)

Entity	Fields (Type)	Description
Order	id: UUID, storeId: UUID, createdAt: datetime, status: enum, value: number, codAmount: number, address: string, pincode: string, courier: string	Customer order data. Tracks status and COD.
ImportJob	id: UUID, fileName: string, startedAt: datetime, status: enum, rowCount: int	CSV import logs.
NdrCase	id: UUID, orderId: UUID, attempts: int, reason: string, status: enum, reportedAt: datetime	A failed delivery incident.
ActionItem	id: UUID, orderId: UUID, type: enum (Call/Message/Hold/Cancel), assignedTo: UUID, status: enum, dueAt: datetime	Task in daily queue.
SavingsEvent	id: UUID, orderId: UUID, type: enum, amountSaved: number, timestamp: datetime	Logged avoided loss (for reporting).
PincodeStats	pincode: string, period: date, totalOrders: int, codOrders: int, rtoOrders: int	Aggregated stats for reporting.
CourierStats	courier: string, period: date, totalOrders: int, rtoOrders: int	Aggregated stats per courier.
PrepaidOpportunity	id: UUID, orderId: UUID, incentive: number, expectedUplift: number	Suggested COD→prepaid conversion.
User	id: UUID, brandId: UUID, email: string, name: string, role: enum, pwdHash: string	Platform user accounts.
Brand	id: UUID, name: string, settings: JSON	D2C brand profile (pricing, fees, etc.).
Store	id: UUID, brandId: UUID, platform: string, credentials: JSON	Connected store info (Shopify, etc.).

Table 4. API Endpoints & Event Types

Endpoint	Method	Description	Event Emitted
POST /api/import	POST	Upload orders CSV	ImportCompleted
GET /api/orders	GET	List/filter orders	–
PATCH /api/orders/{id}	PATCH	Update order fields (status, etc.)	OrderUpdated
GET /api/actions	GET	List pending action items	–
POST /api/actions/{id}/complete	POST	Mark action done (or skipped)	ActionCompleted
GET /api/ndr	GET	List NDR cases	–

POST /api/ndr/{id}/resolve Endpoint	POST Method	Mark NDR as resolved/delivered Description	NdrResolved Event Emitted
GET /api/savings	GET	View savings ledger entries	—
GET /api/dashboard	GET	Profit Cockpit data (aggregated KPIs)	—

Event Type	Description (Payload)
OrderImported	Triggered after new orders imported (payload: import summary)
OrderCreated	Each new order (id, COD flag, value, etc.)
OrderUpdated	Order status change (shipped, delivered, etc.)
NdrCreated	Courier reports failed delivery (orderId, reasonCode)
NdrResolved	NDR resolved (delivered on retry or customer canceled)
ActionCreated	New task added to queue (orderId, actionType)
ActionCompleted	Task completed (including outcome, e.g. rescued=true)
SavingsLogged	A savings event was recorded (amountSaved, type)

These endpoints and events form the REST/async contract between UI and services. For instance, when a CSV is imported (`POST /api/import`), the `OrderImported` event seeds downstream workflows (risk scoring and statistics). Services subscribe to events rather than directly calling each other. This enables features to be enabled/disabled without tightly coupling flows.

Notes on WhatsApp & Courier: RTOShield uses WhatsApp templates for communication (charging ₹0.1150/message in India 【10†L230-L237】 , and free service messages within 24h). Courier integrations obey each courier’s NDR API format. We assume no special courier constraints beyond normal API limits.

Sources: Data on RTO/COD impacts and costs are drawn from recent e-commerce logistics analyses 【22†L118-L121】 【14†L146-L154】 . WhatsApp Business pricing is confirmed for India (Jan 2026) 【10†L230-L237】 . Architecture principles are aligned with industry best-practices for microservices 【18†L310-L319】 . The solution design emphasizes modularity and toggling per feature flag guidance 【24†L77-L82】 . Testing guidance follows modern microservices testing strategies 【29†L121-L129】 . Privacy compliance follows India’s DPDP draft rules 【27†L533-L541】 .